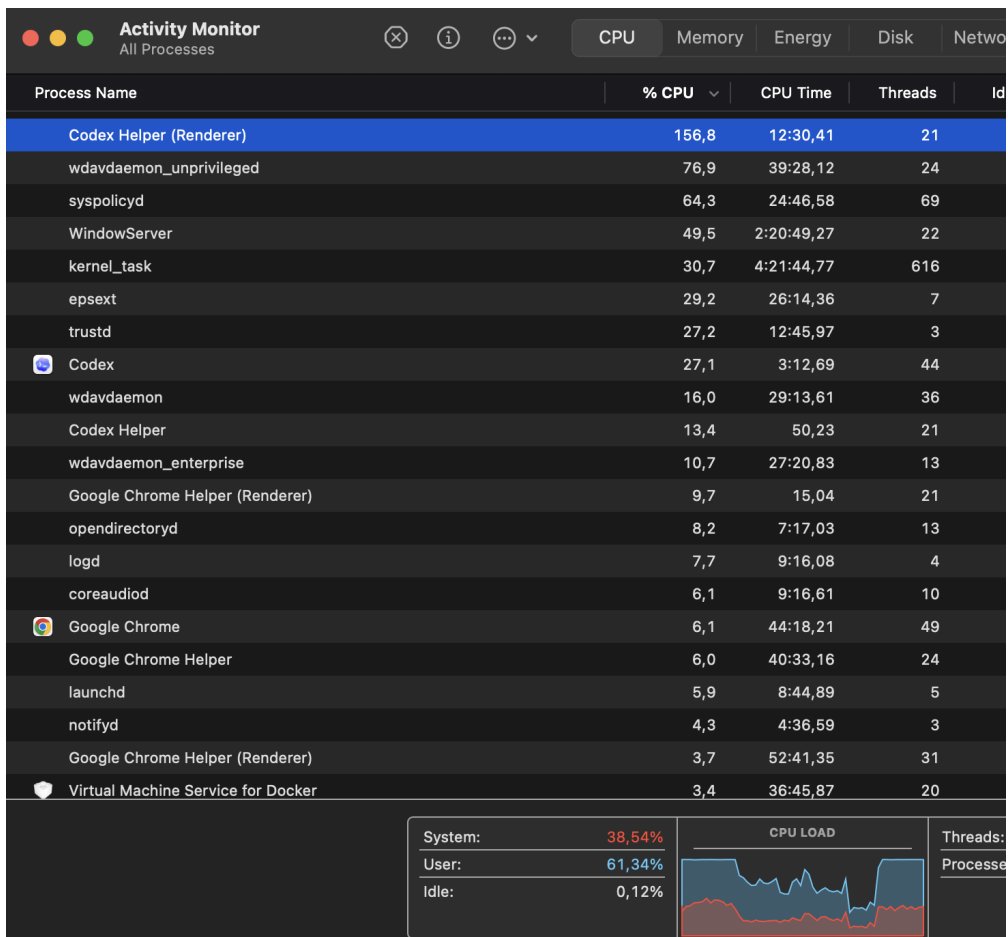


Operating Systems & Processes

DSC 204A: Scalable Data Systems

Haojian Jin



←

r/codex · 16h ago

SlopTopZ

...

Codex eating 10GB of RAM - fix your Electron garbage

Complaint

just got a system out of memory warning and saw this

Codex: 10.29 GB

10 gigabytes. for a coding assistant. on Electron.

please fix this, it's genuinely embarrassing for a product at this level

↑ 7 ↓

💬 11

🔔

🔗 Share

🌀 OpenAI · Promoted

...

Ready to build your next project? Start now with Codex, available with ChatGPT.

Sign Up

chatgpt.com

Codex

There's coding with AI, and then there's Codex

Foundation of Data Systems

- Computer Organization
 - Representation of data
 - Processors, memory, storage
- **OS basics**
 - Process, scheduling
 - Multi-Agent Scheduling
 - Agent team

What is an Operating System?

- Layers between applications and hardware
- OS makes computer hardware useful to programmers
 - Otherwise, users need to speak machine code to computer
- **[Usually] Provides abstractions for applications**
 - Manages and hides details of hardware
 - Accesses hardware through low-level interfaces unavailable to applications
- **[Often] Provides protection**
 - Prevents one app/user from clobbering another

Hardware → **OS** → Apps

A Primitive OS v1

- **OS v1**: just a library of standard services [no protection]
- Simplifying assumptions:
 - System runs one program at a time
 - No bad users or programs (?)
- **Problem: poor utilization**
 - Poor utilization of hardware (e.g., CPU idle while waiting for disk)
 - Poor utilization of human user (must wait for each program to finish)

OS v2: Multi-tasking

- Extend the OS to support many apps
 - When one process blocks (waiting for disk, network, user input, etc.) run another process
- **Problem: What can an ill-behaved process do?**
 - Go into infinite loop and never relinquish CPU
 - Scribble over other processes' memory to make them fail
- **OS provides protection mechanisms:**
 - **Preemption** – take CPU away from looping process
 - **Memory protection** – protect one process' memory from another

What is a Real OS?

- OS: manage and assign hardware resources to apps
- **Goal:** with N users/apps, system not N times slower
 - **Idea:** Giving resources to users who actually need them
- What can go wrong?
 - **One app can interfere with another** (need isolation)
 - **Users are gluttons, use too much CPU** (need scheduling)
 - **Total memory of all apps > machine's RAM** (need memory management)
 - **Disks are shared across apps** (need file systems)

Summary of OS

A software between apps and hardware:

- **Goal 1:** Provide convenience to users
- **Goal 2:** Efficiency – manage compute, memory, storage resources
 - Goal 2.1: Running N processes **not** N times slower
 - Goal 2.2: Running N apps even when total memory $\gg$ physical memory cap
- **Goal 3:** Provide protection
 - One process won't mess up the entire computer
 - One process won't mess up other processes

Key mechanisms: **System calls, Process management, Memory management**

OS provides Isolation using System Calls

System call: The layer for isolation – it abstracts the hardware and provides APIs for programs to use

Kernel Component	Functionality
Process Management	Virtualize processor; "Process" abstraction
Main Memory Management	Virtualize main memory
Filesystems	Virtualize disks; "File" abstraction
Device Drivers	Talk to other I/O devices
Networking	Communication over network

Hardware device-specific programs sit below the kernel.

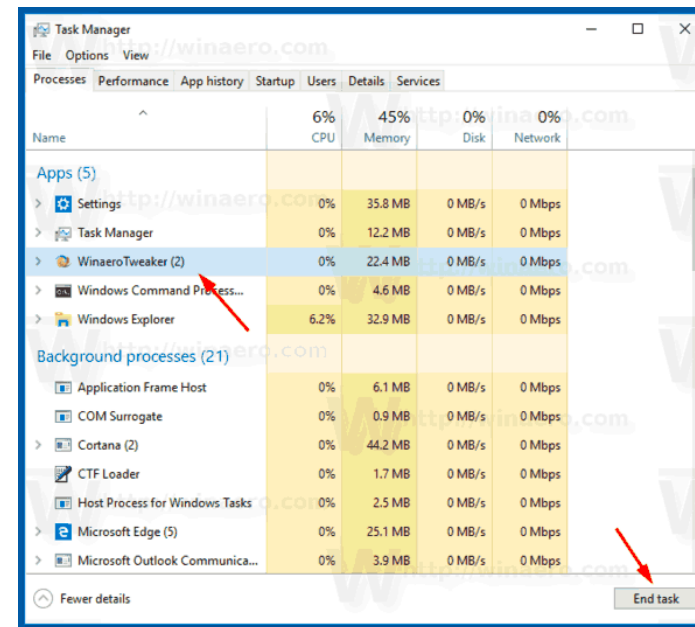
Processes

Foundation of Data Systems

- Computer Organization
 - Representation of data
 - Processors, memory, storage
- OS basics
 - **Process, scheduling**
 - Memory

Processes — the central abstraction in OS

- **Definition:** A process is an instance of a running program
 - One of the most profound ideas in computer science
 - Not the same as "program" or "processor"



Processes — key abstractions

Process provides each program with two key abstractions:

- **Compute Resource**

- Each program seems to have exclusive use of the CPU
- Provided by kernel mechanism called **context switching**

- **Memory Resource**

- Each program seems to have exclusive use of main memory
- Provided by kernel mechanism called **virtual memory**

The Abstraction of a Process

High-level steps OS takes to get a process going:

1. Create a process (get Process ID; add to Process List)
2. Assign part of DRAM to process, aka its **Address Space**
3. Load code and static data (if applicable) to that space
4. Set up the inputs needed to run program's `main()`
5. Update process' State to **Ready**
6. When process is scheduled (**Running**), OS temporarily hands off control to process
7. Eventually, process finishes or is destroyed

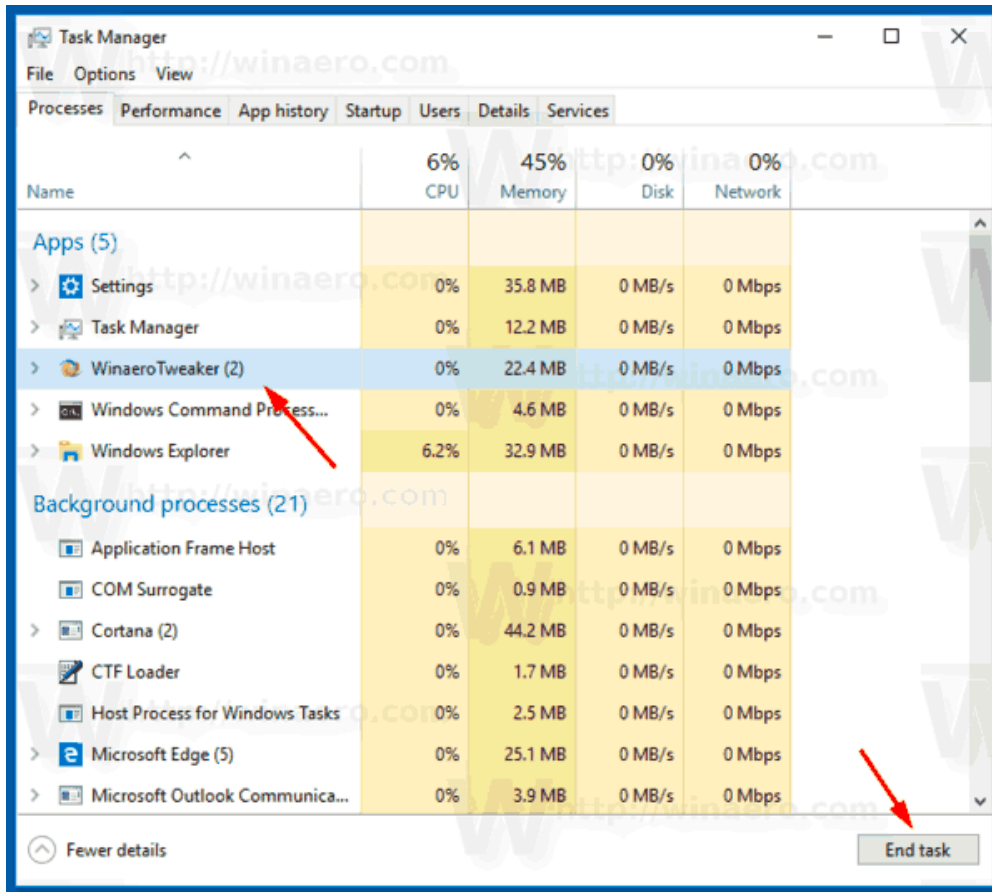
Virtualization of Hardware Resources

- OS has mechanisms and policies to regain control
- **Virtualization**: Each hardware resource is treated as a virtual entity that OS can divvy up among processes
- **Limited Direct Execution**:
 - OS mechanism to time-share CPU and preempt a process to run a different one, aka "**context switch**"
 - A **scheduling policy** tells OS what time-sharing to use
 - Processes must transfer control to OS for "privileged" operations (e.g., I/O);
System Calls API

Q: But isn't it risky for OS to hand off control of hardware to a process (random user-written program)?!

Multiprocessing in OS: The Illusion

Computer runs many processes simultaneously



The screenshot shows the Windows Task Manager window with the 'Performance' tab selected. The top section displays system resource usage: CPU at 6%, Memory at 45%, Disk at 0%, and Network at 0%. Below this is a list of running processes, categorized into 'Apps (5)' and 'Background processes (21)'. A red arrow points to the 'WinaeroTweaker (2)' process in the 'Apps' list. Another red arrow points to the 'End task' button at the bottom right of the window.

Name	CPU	Memory	Disk	Network
Apps (5)				
Settings	0%	35.8 MB	0 MB/s	0 Mbps
Task Manager	0%	12.2 MB	0 MB/s	0 Mbps
WinaeroTweaker (2)	0%	22.4 MB	0 MB/s	0 Mbps
Windows Command Process...	0%	4.6 MB	0 MB/s	0 Mbps
Windows Explorer	6.2%	32.9 MB	0 MB/s	0 Mbps
Background processes (21)				
Application Frame Host	0%	6.1 MB	0 MB/s	0 Mbps
COM Surrogate	0%	0.9 MB	0 MB/s	0 Mbps
Cortana (2)	0%	44.2 MB	0 MB/s	0 Mbps
CTF Loader	0%	1.7 MB	0 MB/s	0 Mbps
Host Process for Windows Tasks	0%	2.5 MB	0 MB/s	0 Mbps
Microsoft Edge (5)	0%	25.1 MB	0 MB/s	0 Mbps
Microsoft Outlook Communica...	0%	3.9 MB	0 MB/s	0 Mbps

Multiprocessing Example

```
xterm
Processes: 123 total, 5 running, 9 stuck, 109 sleeping, 611 threads
Load Avg: 1.03, 1.13, 1.14 CPU usage: 3.27% user, 5.15% sys, 91.56% idle
SharedLibs: 576K resident, 0B data, 0B linkedit.
MemRegions: 27958 total, 1127M resident, 35M private, 494M shared.
PhysMem: 1039M wired, 1974M active, 1062M inactive, 4076M used, 18M free.
VM: 280G vsize, 1091M framework vsize, 23075213(1) pageins, 5843367(0) pageouts.
Networks: packets: 41046228/11G in, 66083096/77G out.
Disks: 17874391/349G read, 12847373/594G written.

PID    COMMAND    %CPU TIME    #TH    #WQ    #PORT    #MREG    RPRVT    RSHRD    RSIZE    VPRVT    VSIZE
99217-  Microsoft Of 0.0 02:28.34 4      1      202     418     21M     24M     21M     66M     763M
99051  usbmuxd    0.0 00:04.10 3      1      47      66     436K    216K    480K    60M     2422M
99006  iTunesHelper 0.0 00:01.23 2      1      55      78     728K    3124K   1124K   43M     2429M
84286  bash       0.0 00:00.11 1      0      20      24     224K    732K    484K    17M     2378M
84285  xterm      0.0 00:00.83 1      0      32      73     656K    872K    692K    9728K   2382M
55939-  Microsoft Ex 0.3 21:58.97 10     3      360     954     16M     65M     46M     114M    1057M
54751  sleep      0.0 00:00.00 1      0      17      20     92K     212K    360K    9632K   2370M
54739  launchdadd 0.0 00:00.00 2      1      33      50     488K    220K    1736K   48M     2409M
54737  top        6.5 00:02.53 1/1    0      30      29     1416K   216K    2124K   17M     2378M
54719  automountd 0.0 00:00.02 7      1      53      64     860K    216K    2184K   53M     2413M
54701  ocspd      0.0 00:00.05 4      1      61      54     1268K   2644K   3132K   50M     2426M
54661  Grab       0.6 00:02.75 6      3      222+    389+    15M+    26M+    40M+    75M+    2556M+
54659  cookied    0.0 00:00.15 2      1      40      61     3316K   224K    4088K   42M     2411M
53818  mdworker   0.0 00:01.67 4      1      52      91     7628K   7412K   16M     48M     2438M
50878  mdworker   0.0 00:11.17 3      1      53      91     2464K   6148K   9976K   44M     2434M
50410  xterm      0.0 00:00.13 1      0      32      73     280K    872K    532K    9700K   2382M
50078  emacs      0.0 00:06.70 1      0      20      35     52K     216K    88K     18M     2392M
```

top command in terminal: many processes, identified by Process ID (PID)

Multiprocessing: A Strawman Solution

- Assign individual memory (say 1/3) to each app
- Assign CPU to work on an app until completion → then next

	G1: Convenient?	G2: Efficient?	G3: Protection?
Answer	Yes	No!	Yes

We are N times slower when running N processes!

Multiprocessing: Time Sharing of Processors

Idea: Virtualize the CPU time as **time slices**; assign time slices to different processes

1. Process 1 runs at $t = 1$
2. **Save** current registers in memory
3. Assign time slice $t = 2$ to the next process
4. **Resume** progress: move saved registers from memory to CPU
5. Repeat — this is called **context switch**

Multiprocessing: Multiple Processors

- Multiple CPU cores?
- All processors sweep from left (1st process) to right (last process)
- Each processor accounts for $\frac{1}{2}$ of the processes

Let's Implement It!

Physical Processor → OS's virtualized
CPU abstraction → PID1, PID2, PID3, ...

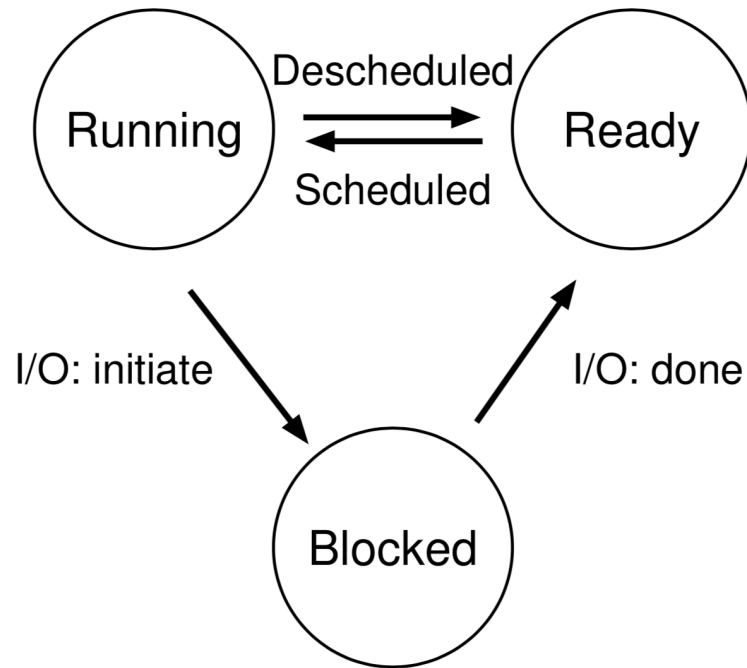
**GAP1: How to virtualize CPU
resources temporally and
spatially?**



Scheduling

Process State and CPU Time

OS keeps moving processes between 3 states:



Gantt Chart: A visualization to show what process runs when (on processor)

P1	Idle	P2	P1	P2	...
-----------	-------------	-----------	-----------	-----------	------------

Scheduling Policies/Algorithms

- **Schedule:** Record of what process runs on each CPU when
- **Policy** controls how OS time-shares CPUs among processes
- Key terms for a process (aka job):
 - **Arrival Time:** Time when process gets created
 - **Job Length:** Duration of time needed for process
 - **Start Time:** Time when process first starts on processor
 - **Completion Time:** Time when process finishes/killed
 - **Response Time** = Start Time – Arrival Time
 - **Turnaround Time** = Completion Time – Arrival Time
- **Workload:** Set of processes, arrival times, and job lengths

Scheduling Policy: FIFO

First-In-First-Out aka First-Come-First-Serve (FCFS)

Ranking criterion: Arrival Time; no preemption allowed

Example: P1, P2, P3 of lengths 10, 40, 10 units arrive closely in that order

Process	Arrival Time	Start Time	Completion Time	Response Time	Turnaround Time
P1	0	0	10	0	10
P2	0	10	50	10	50
P3	0	50	60	50	60
			Avg:	20	40

Main con: Short jobs may wait a lot, aka "**Convoy Effect**"

Scheduling Policy: SJF

Shortest Job (next) First

Ranking criterion: Job Length; no preemption allowed

Example: P1, P2, P3 of lengths 10, 40, 10 units arrive closely in that order

Process	Arrival Time	Start Time	Completion Time	Response Time	Turnaround Time
P1	0	0	10	0	10
P2	0	20	60	20	60
P3	0	10	20	10	20
			Avg:	10	30

Cons: Not all job lengths known beforehand; long processes may be held off indefinitely

(FIFO Avg: 20 and 40)

Example Exam Q1: Round Robin Schedule

- RR does **not** need to know job lengths
- Fixed **time quantum** given to each job; cycle through jobs

Example: P1, P2, P3 of lengths 10, 40, 10 units; quantum = 5

P1	P2	P3	P1	P2	P3	P2	P2	P2	P2	P2	P2
0	5	10	15	20	25	30	35	40	45	50	55

RR is often very fair, but **Avg Turnaround Time goes up!**

Example Exam Q2: SCTF

Shortest Completion Time First

- Jobs might not all arrive at same time; **preemption possible**

Example: P1, P2, P3 of lengths 10, 40, 10 units arrive at different times

P2	P1	P2	P3	P2	P2	P2
0	10	20	25	35	45	55

- P1 arrives → switch
- P3 arrives → switch

Scheduling Policies/Algorithms – Summary

- In general, not all Arrival Times and Job Lengths are known beforehand. But preemption is possible.
- **Key Principle:** Inherent tension between overall workload **performance** and allocation **fairness**
 - Performance metric is usually Average Turnaround Time
 - Many fairness metrics exist, e.g., Jain's fairness index
- 100s of scheduling policies studied: FIFO, SJF, STCF, Round Robin, Random, etc.
 - Different criteria for ranking; preemptive vs not
 - Complex "multi-level feedback queue" schedulers
 - ML-based schedulers are "hot" nowadays!

Scheduling in ChatGPT



- What is the **response time**?
- What is the **turnaround time**?
- What is **fairness**?
- Do we know the **job length**?
- Can we run S1/S2/S3 together?
- **How to schedule?**